

Programmieren für Kids

JavaScript für Kids

Modul 4

Schleifen: Dinge viele Male tun



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Deine Programme können schon viel. Aber jeden Befehl schreibst du bisher **einzeln**. Stell dir vor, du sollst die Zahlen von 1 bis 100 ausgeben. Von Hand?

Hundert Zeilen! Heute lernst du den besseren Weg: etwas viele Male tun, ohne alles abzutippen.



Das baust du in diesem Modul

- eine **Einmaleins-Maschine**, die eine ganze Reihe zeigt
- eine **Bestenliste**, die alle Namen auflistet
- ein **Zahlen-Rätsel**, bei dem du gegen den Computer rätst

Schleifen sind der Grund, warum Computer riesige Arbeit in Sekunden schaffen.



Kapitel 1

Die for-Schleife

Warum Schleifen?

Ohne Schleife müsstest du dasselbe immer wieder tippen:

- 100 Zahlen ausgeben = 100 Zeilen
- mit einer Schleife = **eine** Zeile

Eine Schleife wiederholt einen Block so oft, wie du willst.

Die for-Schleife

```
for (let i = 1; i <= 5; i++) {  
  console.log(i)  
}
```

In der Konsole erscheint:

```
1 2 3 4 5
```

Die drei Teile

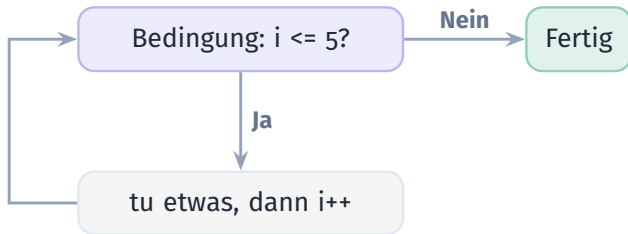
In den runden Klammern stehen drei Teile, mit Strichpunkt getrennt:

let i = 1 = der Start (bei 1 beginnen)

i <= 5 = die Bedingung (solange)

i++ = der Schritt (eins hoch)

So läuft eine Schleife



Prüfen, ausführen, hochzählen, wieder prüfen. Bis die Bedingung nicht mehr stimmt.

Etwas mehrmals tun

Der Zähler muss nicht angezeigt werden. Oft willst du einfach dasselbe mehrmals:

```
for (let i = 1; i <= 3; i++) {  
  console.log("Hurra!")  
}
```

Willst du es hundertmal? Ändere die 3 in 100. Eine Zeile statt hundert.

Eine Zeile in jeder Runde bauen

Ein wichtiger Trick: in jeder Runde etwas an einen Text **anhängen**. `<br>` macht dabei einen Zeilenumbruch auf der Seite.

```
let text = ""
for (let i = 1; i <= 3; i++) {
  text = text + "Zeile " + i + "<br>"
}
document.getElementById("tafel").innerHTML = text
```

Die Kiste text startet leer und wächst in jeder Runde.

Übung: die Schleife zählt



ÜBUNG

ca. 10 Minuten

Lass den Computer von 1 bis 10 zählen:

```
for (let i = 1; i <= 10; i++) {  
  console.log(i)  
}
```

★ **Bonus:** Zähle rückwärts von 10 bis 1 (Tipp: `i--`).

Übung: die Mal-Reihe



ÜBUNG

ca. 10 Minuten

Lass den Computer die 3er-Reihe ausgeben (3, 6, 9 ...):

```
for (let i = 1; i <= 10; i++) {  
  console.log(i * 3)  
}
```

★ **Bonus:** Gib nur die geraden Zahlen bis 20 aus (Tipp: `i = i + 2`).

Fehler-Detektive



FINDE DEN FEHLER

Diese Schleife hört nie auf, der Browser hängt. Warum?

```
for (let i = 1; i <= 5; ) {  
  console.log(i)  
}
```

Tipp: Was macht den Zähler größer?



Projekt

Die Einmaleins-Maschine



Jetzt öffnen: [4a-einmaleins](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<div id="tafel"></div>  
<button onclick="einmaleins()">Zeig das Einmaleins</button>
```

Ein leeres Feld namens **tafel** und ein Knopf, der `einmaleins()` ruft.

Der Code

```
const reihe = 7    // ändere das auf deine Lieblingszahl

function einmaleins() {
  let text = ""
  for (let i = 1; i <= 10; i++) {
    text = text + i + " x " + reihe + " = " + (i * reihe) +
      "<br>"
  }
  document.getElementById("tafel").innerHTML = text
}
```

Die Schleife baut Zeile für Zeile: 1 x 7 = 7, 2 x 7 = 14 ...

Übung: dein Einmaleins



PROJEKT

ca. 12 Minuten

- Bring die Einmaleins-Maschine zum Laufen
- Ändere `reihe` auf deine Lieblingszahl

★ **Bonus:** Frag mit `prompt`, welche Reihe: `reihe = Number(prompt("Welche Reihe?"))`.

Zeig deine Maschine!

$1 \times 7 = 7$
 $2 \times 7 = 14$
 $3 \times 7 = 21$
...

Zeig das Einmaleins



Kapitel 2

Durch eine Liste gehen

¹₂₃ ≡ Jedes Ding der Reihe nach

In Modul 3 hast du Listen kennengelernt. Mit einer Schleife gehst du durch **alle** Dinge:

```
const namen = ["Mia", "Tom", "Lena"]

for (let i = 0; i < namen.length; i++) {
  console.log(namen[i])
}
```

Der Zähler startet bei 0 und läuft, solange er kleiner als `.length` ist.

Die Standard-Schleife über eine Liste

```
for (let i = 0; i < liste.length; i++)
```

Diese Zeile brauchst du immer wieder. In der Schleife ist `liste[i]` das aktuelle Ding, egal wie lang die Liste ist.

Übung: durch eine Liste laufen



ÜBUNG

ca. 10 Minuten

Baue eine Liste mit Zahlen und gib von jeder das **Doppelte** aus:

```
const zahlen = [3, 7, 10]

for (let i = 0; i < zahlen.length; i++) {
  console.log(zahlen[i] * 2)
}
```

★ **Bonus:** Gib stattdessen alle Zahlen + 1 aus.

Übung: alles zusammenzählen



ÜBUNG

ca. 10 Minuten

Zähle alle Zahlen zusammen. Die summe wächst in jedem Durchgang:

```
const zahlen = [5, 8, 2]
let summe = 0
for (let i = 0; i < zahlen.length; i++) {
  summe = summe + zahlen[i]
}
console.log(summe)    // 15
```



Projekt

Die Bestenliste



Jetzt öffnen: [4b-bestenliste](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<div id="liste"></div>  
<button onclick="zeigeListe()">Zeig die Bestenliste</button>
```

Ein leeres Feld namens **liste** und ein Knopf, der `zeigeListe()` ruft.

Der Code

```
const namen = ["...", "...", "..."] // deine Namen

function zeigeListe() {
  let text = ""
  for (let i = 0; i < namen.length; i++) {
    text = text + (i + 1) + ". " + namen[i] + "<br>"
  }
  document.getElementById("liste").innerHTML = text
}
```

(i + 1) für die Nummer: Menschen zählen ab 1, der Computer ab 0.

Fehler-Detektive



FINDE DEN FEHLER

Am Ende der Liste erscheint „undefined“. Warum?

```
const namen = ["Mia", "Tom"]
for (let i = 0; i <= namen.length; i++) {
  console.log(namen[i])
}
```

Tipp: Die letzte Position ist `length - 1`. Ist `<=` richtig?

Übung: deine Bestenliste



PROJEKT

ca. 12 Minuten

- Bring die Bestenliste zum Laufen
- Schreibe deine eigenen Namen in die Liste

★ **Bonus:** Mach eine Playlist deiner Lieblingslieder oder eine Einkaufsliste.



Zeig deine Bestenliste!

1. Mia
2. Tom
3. Lena
4. Ben

Zeig die Bestenliste



Kapitel 3

Solange wiederholen

Die while-Schleife

Die for-Schleife zählt bis zu einer festen Zahl. Manchmal weißt du aber nicht, wie oft. Dafür gibt es `while`:

```
let count = 1

while (count <= 3) {
  console.log(count)
  count++
}
```

`while` wiederholt, solange die Bedingung stimmt.

Übung: die while-Schleife



ÜBUNG

ca. 10 Minuten

Zähle mit `while` von 1 bis 5:

```
let i = 1
while (i <= 5) {
  console.log(i)
  i = i + 1
}
```

★ **Bonus:** Lass sie bei 10 starten und rückwärts bis 0 zählen.



Vorsicht: Endlosschleife

Ändert sich in der Schleife nichts, das die Bedingung falsch macht, läuft sie **ewig** weiter, und der Browser hängt.



Frag dich immer

Was macht die Bedingung irgendwann falsch? (Hier: `count++`)



Projekt

Das Zahlen-Rätsel



Jetzt öffnen: `4c-zahlen-raetsel`

in VS Code – oder auf der Kursseite



So funktioniert es

Der Computer denkt sich eine Zahl aus, und du rätst, bis du sie triffst.

- nach jedem Versuch sagt er „zu klein“ oder „zu groß“
- du wiederholst, **bis** du richtig liegst

Genau ein Fall für `while`: du weißt vorher nicht, wie viele Versuche du brauchst.

Der Code

```
const zahl = Math.floor(Math.random() * 100) + 1
let tipp = Number(prompt("Rate eine Zahl von 1 bis 100:"))
while (tipp !== zahl) {
  if (tipp < zahl) {
    tipp = Number(prompt("Zu klein! Nochmal:"))
  } else {
    tipp = Number(prompt("Zu gross! Nochmal:"))
  }
}
alert("Richtig! Die Zahl war " + zahl)
```

`!==` heißt „nicht gleich“, `Number(...)` macht aus Text eine Zahl.

Fehler-Detektive



FINDE DEN FEHLER

Dieses Rätsel hört nie auf, auch wenn du richtig rätst. Warum?

```
let tipp = prompt("Rate:")

while (tipp !== zahl) {
  tipp = prompt("Nochmal:")
}
```

Tipp: prompt liefert Text. Ist der Text „42“ gleich der Zahl 42?

Übung: dein Zahlen-Rätsel



PROJEKT

ca. 15 Minuten

- Bring das Rätsel zum Laufen und spiel ein paar Runden
- Ändere den Zahlenbereich (z.B. 1 bis 1000)

★ **Bonus:** Zähle mit, wie viele Versuche du gebraucht hast, und zeig es am Ende an.



Projekt

Monster-Wellen



Jetzt öffnen: [4d-monster-wellen](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="monster"></h2>
<p id="text">Hau auf das Monster!</p>
<button onclick="schlag()">Schlag zu!</button>
```

Ein Feld **monster** für den Lebensbalken, eine Nachricht und ein Knopf, der `schlag()` ruft.

Der Lebensbalken mit einer Schleife

Erinnerst du dich ans Zeilen-Bauen? Genauso bauen wir einen Balken aus # – einer pro Leben:

```
let welle = 1
let monsterLeben = 10

function zeigeMonster() {
  let balken = ""
  for (let i = 0; i < monsterLeben; i++) {
    balken = balken + "#"
  }
  document.getElementById("monster").innerHTML = "Welle " + welle +
    ": " + balken
}
```

Zuschlagen

Jeder Klick nimmt dem Monster ein Leben. Ist es besiegt, kommt eine stärkere Welle:

```
function schlag() {  
  monsterLeben = monsterLeben - 1  
  if (monsterLeben <= 0) {  
    welle = welle + 1  
    monsterLeben = 10 + welle * 2  
    document.getElementById("text").innerHTML = "Neue Welle: " +  
    welle  
  }  
  zeigeMonster()  
}
```

Übung: dein Monster-Wellen-Spiel



PROJEKT

ca. 15 Minuten

- Bring das Spiel zum Laufen (Balken, Zuschlagen, neue Welle)
- Ändere, wie stark die Monster pro Welle werden

★ **Bonus:** Mach pro Klick 2 Schaden statt 1 (`monsterLeben - 2`).



Zeig dein Monster-Spiel!

Welle 3

#

Schlag zu!

✓ Das kannst du jetzt

- mit einer `for`-Schleife etwas viele Male tun
- durch eine ganze **Liste** gehen
- mit `while` wiederholen, bis etwas stimmt
- eine Endlosschleife erkennen und vermeiden

Mit Schleifen schaffst du in einer Zeile, wofür du sonst hundert bräuchtest.

☰ Neue Befehle auf einen Blick

for (...)	=	festе Anzahl wiederholen
while (...)	=	wiederholen, bis es stimmt
i++	=	den Zähler hochzählen
liste.length	=	Länge einer Liste
!==	=	ist nicht gleich

	=	Zeilenumbruch auf der Seite



Als Nächstes: Modul 5

Du hast jetzt alle Bausteine: Variablen, Funktionen, Entscheidungen, Zufall und Schleifen.

Als Nächstes baust du ein großes eigenes Spiel und machst es schön.

Der Rest ist deine Fantasie.